

1. Import Libraries

```
# =====
# 1. IMPORT LIBRARIES
# =====

import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    precision_recall_curve,
    average_precision_score
)

# Visualization settings
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
```

Insight : The required Python libraries were imported for data manipulation, exploratory data analysis, visualization, preprocessing, model training, and classification evaluation. Logistic Regression was selected as the primary predictive model because the target variable represents a binary outcome: whether a user clicked the advertisement or not.

2. Load Dataset

```
# =====
# 2. LOAD DATASET
# =====

df = pd.read_csv("/content/advertising.csv")

print("Dataset Shape:", df.shape)

display(df.head())
```

Dataset Shape: (1000, 10)

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Male	Country	Timestamp	Clicked on Ad
0	68.95	35	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	0	Tunisia	2016-03-27 00:53:11	0
1	80.23	31	68441.85	193.77	Monitored national standardization	West Jodi	1	Nauru	2016-04-04 01:39:02	0
2	68.47	28	58785.84	222.50	Organic bottom-line	Reedham	0	San	2016-03-13	0

Insight:

The dataset contains historical information about users exposed to online advertisements. The observations include demographic attributes, website engagement behavior, internet usage information, and the final advertisement-click outcome.

3. Dataset Information

```
# =====
# 3. DATASET INFORMATION
# =====

print("Dataset Information:\n")
df.info()
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   Daily Time Spent on Site              1000 non-null   float64
1   Age                                    1000 non-null   int64
2   Area Income                           1000 non-null   float64
3   Daily Internet Usage                  1000 non-null   float64
4   Ad Topic Line                         1000 non-null   object
5   City                                   1000 non-null   object
6   Male                                   1000 non-null   int64
7   Country                                1000 non-null   object
8   Timestamp                             1000 non-null   object
9   Clicked on Ad                         1000 non-null   int64
dtypes: float64(3), int64(3), object(4)
memory usage: 78.3+ KB
```

4. Statistical Summary

```
# =====
# 4. DESCRIPTIVE STATISTICS
# =====

display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max	
Daily Time Spent on Site	1000.0	65.00020	15.853615	32.60	51.3600	68.215	78.5475	91.43	
Age	1000.0	36.00900	8.785562	19.00	29.0000	35.000	42.0000	61.00	
Area Income	1000.0	55000.00008	13414.634022	13996.50	47031.8025	57012.300	65470.6350	79484.80	
Daily Internet Usage	1000.0	180.00010	43.902339	104.78	138.8300	183.130	218.7925	269.96	
Male	1000.0	0.48100	0.499889	0.00	0.0000	0.000	1.0000	1.00	
Clicked on Ad	1000.0	0.50000	0.500250	0.00	0.0000	0.500	1.0000	1.00	

Insight:

Descriptive statistics provide an initial understanding of the central tendency, variability, and range of numerical variables. This helps identify unusual values, understand feature scales, and determine which variables may require further preprocessing.

5. Missing Values

```
# =====
# 5. MISSING VALUE ANALYSIS
# =====

missing_values = df.isnull().sum()

missing_summary = pd.DataFrame({
    "Missing Values": missing_values,
    "Missing Percentage": (missing_values / len(df)) * 100
})

display(missing_summary)
```

	Missing Values	Missing Percentage	
Daily Time Spent on Site	0	0.0	 
Age	0	0.0	
Area Income	0	0.0	
Daily Internet Usage	0	0.0	
Ad Topic Line	0	0.0	
City	0	0.0	
Male	0	0.0	
Country	0	0.0	
Timestamp	0	0.0	
Clicked on Ad	0	0.0	

Next steps: [Generate code with missing_summary](#)[New interactive sheet](#)

Insight — Missing Value Analysis

No missing values were found in any of the dataset columns. All 10 features, including demographic, behavioral, categorical, timestamp, and target variables, have 0 missing values (0.0%). Therefore, no missing-value imputation or row removal was required, allowing the analysis to proceed without introducing assumptions about unavailable data.

6. Duplicate Check

```
# =====
# 6. DUPLICATE ANALYSIS
# =====

duplicate_count = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)
```

Number of duplicate rows: 0

Insight — Duplicate Value Analysis

No duplicate rows were found in the dataset. The duplicate count is 0, indicating that each observation is unique. Therefore, no duplicate records needed to be removed, helping maintain the integrity of the dataset for further analysis and model training.

7. Target Variable Analysis

```
# =====
# 7. TARGET VARIABLE ANALYSIS
# =====

target_counts = df["Clicked on Ad"].value_counts()

print(target_counts)

plt.figure(figsize=(7, 5))

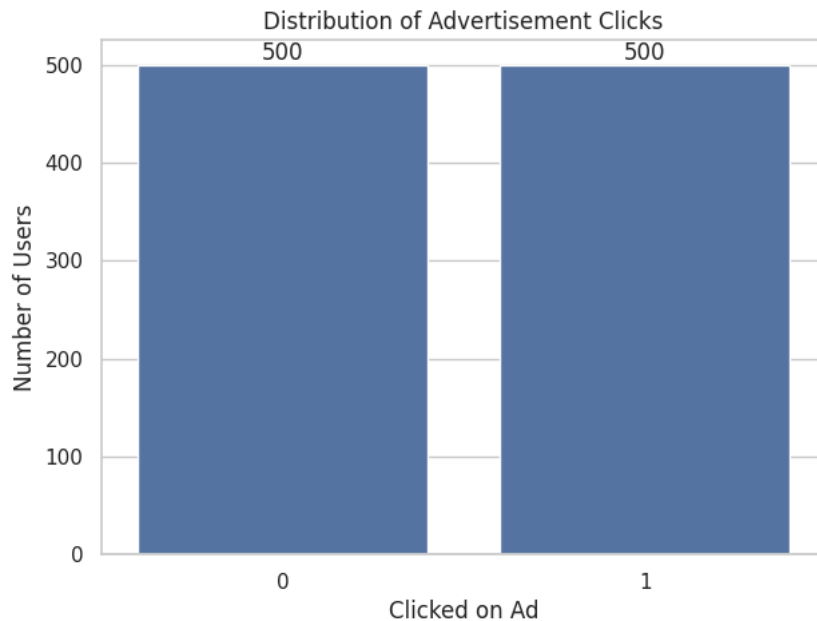
ax = sns.countplot(
    data=df,
    x="Clicked on Ad"
)

plt.title("Distribution of Advertisement Clicks")
plt.xlabel("Clicked on Ad")
plt.ylabel("Number of Users")

for container in ax.containers:
    ax.bar_label(container)

plt.show()
```

```
Clicked on Ad
0    500
1    500
Name: count, dtype: int64
```



Insight — Target Variable Distribution

The target variable Clicked on Ad is perfectly balanced, with 500 users who did not click (0) and 500 users who clicked (1). This means each class represents 50% of the dataset, so there is no class imbalance. Therefore, the model can be evaluated using accuracy alongside precision, recall, F1-score, and ROC-AUC without the results being dominated by an overrepresented class.

```
target_percentage = (
    df["Clicked on Ad"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

print("Target Distribution (%):")
print(target_percentage)
```

```
Target Distribution (%):
Clicked on Ad
0    50.0
1    50.0
Name: proportion, dtype: float64
```

8. Numerical Feature Distributions

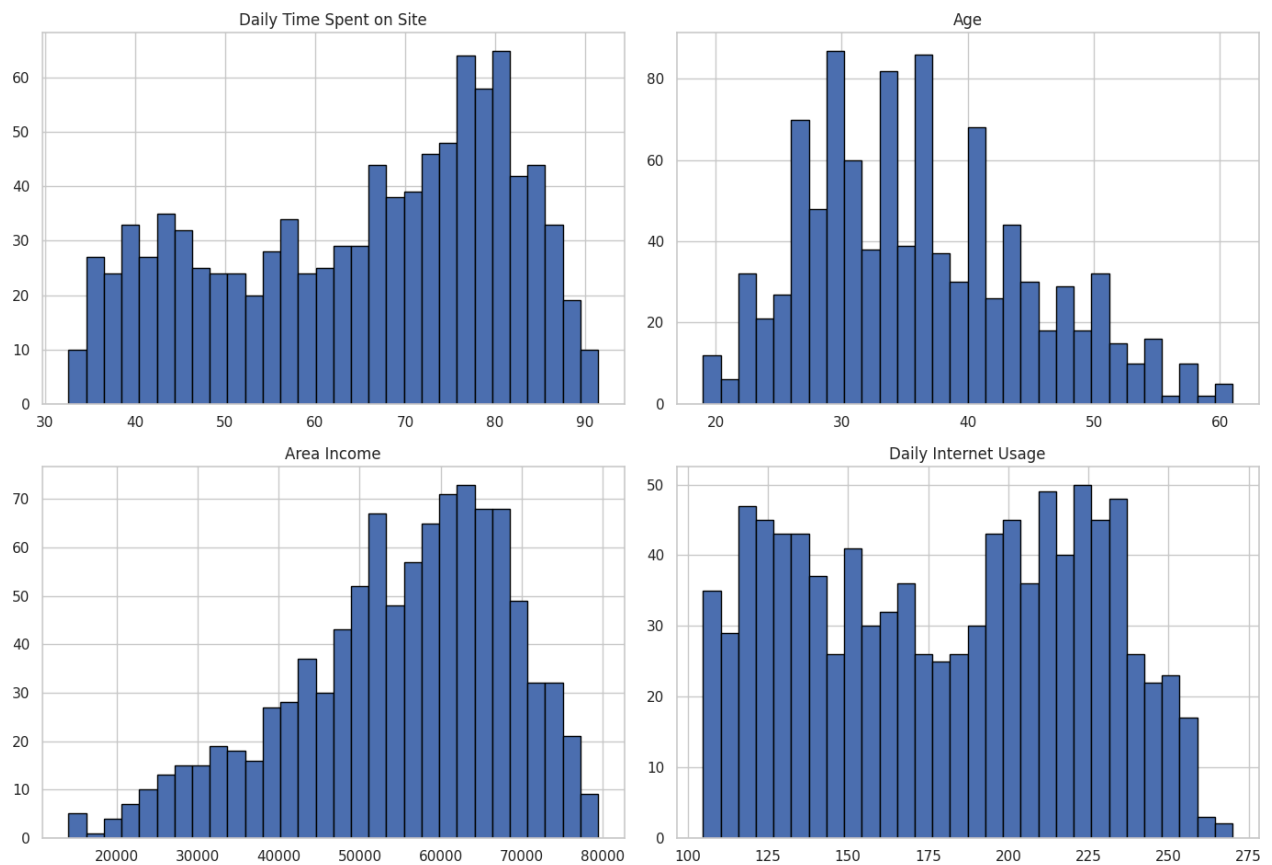
```
# =====
# 8. NUMERICAL FEATURE DISTRIBUTIONS
# =====

numeric_features = [
    "Daily Time Spent on Site",
    "Age",
    "Area Income",
    "Daily Internet Usage"
]

df[numeric_features].hist(
    bins=30,
    figsize=(14, 10),
    edgecolor="black"
)

plt.suptitle("Distribution of Numerical Features", fontsize=16)
plt.tight_layout()
plt.show()
```

Distribution of Numerical Features



Insight

The distribution plots help identify the typical range and spread of user characteristics. They also provide an initial indication of skewness and potential extreme observations that may influence a linear classification model.

9. Boxplots for Outlier Detection

```
# =====
# 9. BOXPLOT ANALYSIS
# =====

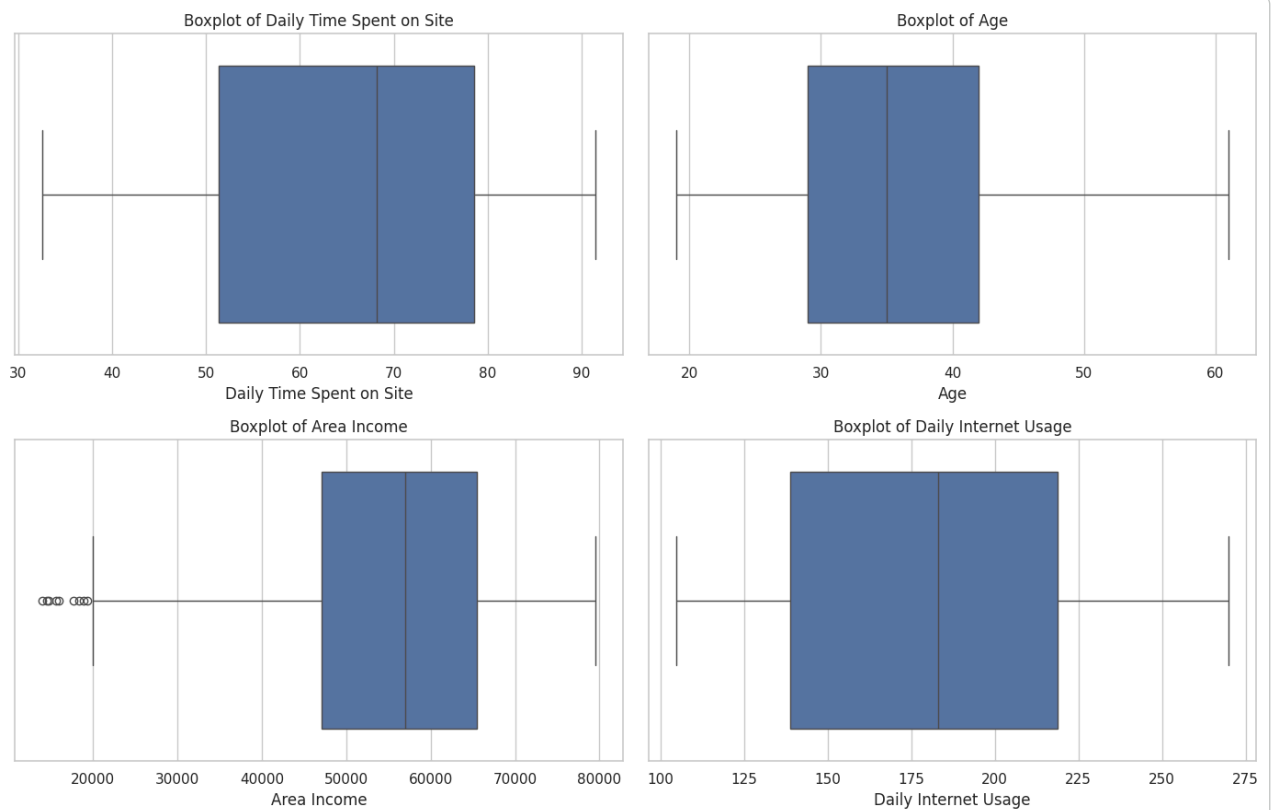
fig, axes = plt.subplots(2, 2, figsize=(14, 9))

for ax, col in zip(axes.flatten(), numeric_features):

    sns.boxplot(
        data=df,
        x=col,
        ax=ax
    )

    ax.set_title(f"Boxplot of {col}")

plt.tight_layout()
plt.show()
```



Insight

Boxplots provide a visual method for identifying observations that fall substantially outside the typical range of each numerical variable. These observations should be investigated before deciding whether they represent genuine users or data-quality issues.

10. Feature vs Target Analysis

```
# =====
# 10. FEATURE VS TARGET ANALYSIS
# =====

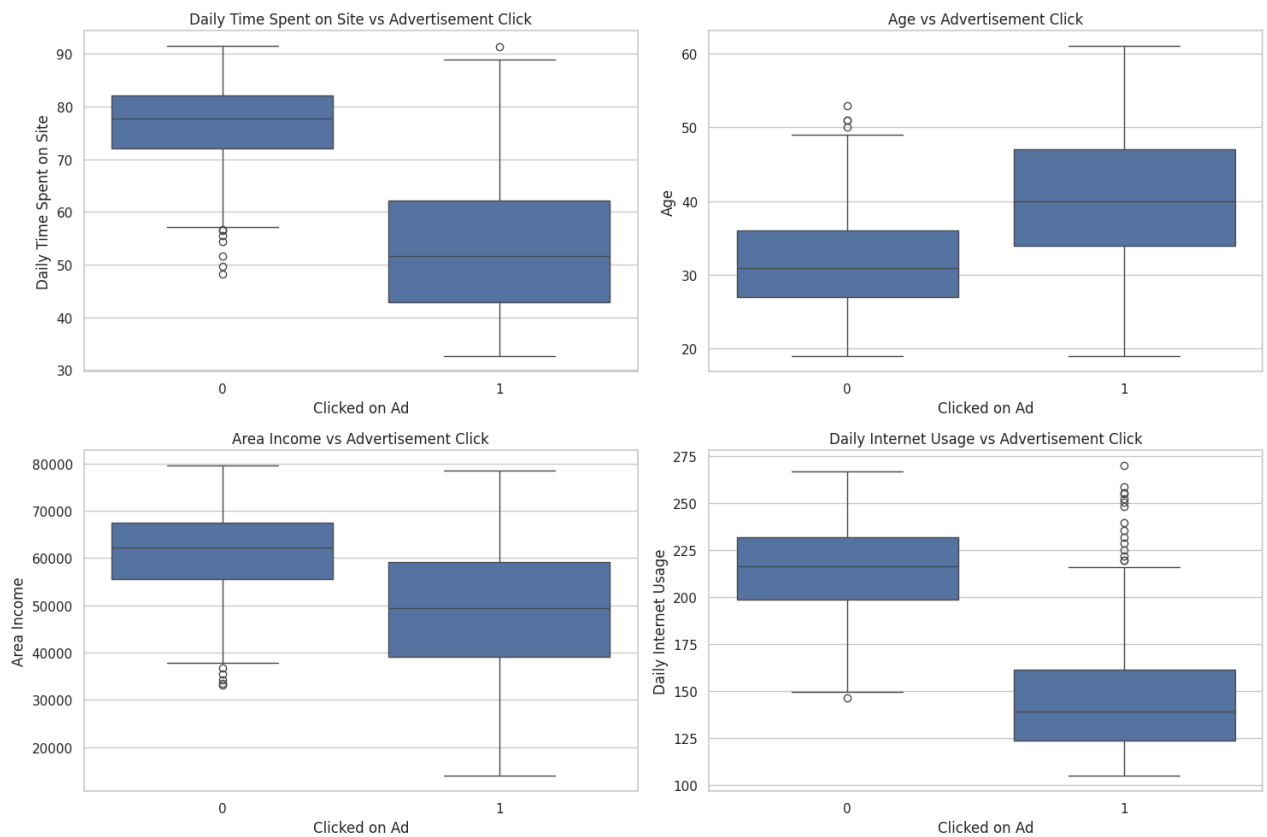
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

for ax, col in zip(axes.flatten(), numeric_features):

    sns.boxplot(
        data=df,
        x="Clicked on Ad",
        y=col,
        ax=ax
    )

    ax.set_title(f"{col} vs Advertisement Click")

plt.tight_layout()
plt.show()
```



Insight

Comparing numerical user characteristics across the two target classes helps identify whether user behavior differs between clickers and non-clickers. Features showing clear distributional differences between the two groups may provide useful predictive information for the classification model.

11. Correlation Heatmap

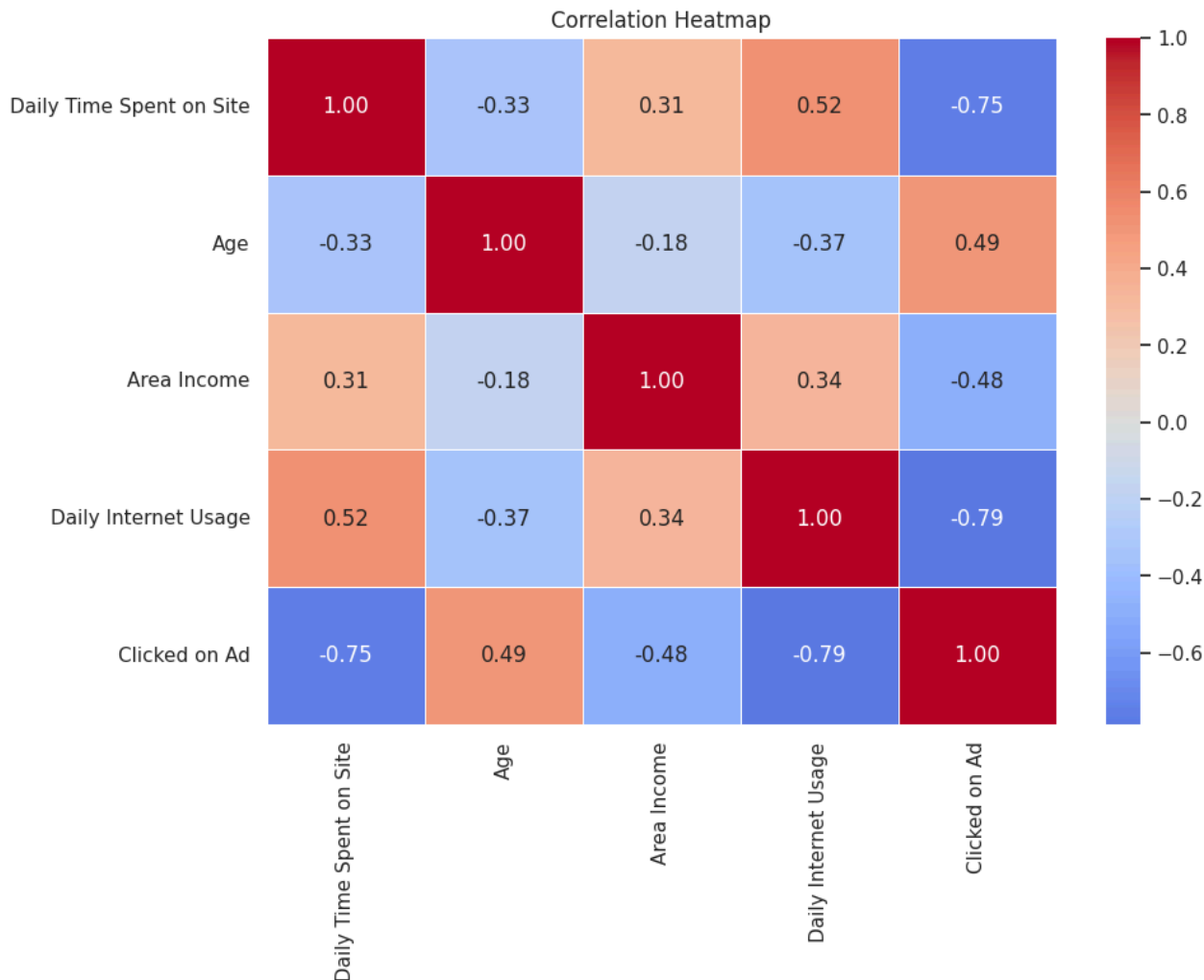
```
# =====
# 11. CORRELATION ANALYSIS
# =====

correlation_matrix = df[numeric_features + ["Clicked on Ad"]].corr()

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    linewidths=0.5
)

plt.title("Correlation Heatmap")
plt.show()
```



Insight

The correlation matrix summarizes the linear relationships between numerical variables and the advertisement-click target. Strong correlations with the target can indicate potentially useful predictive variables, while strong correlations between independent variables may indicate redundancy. However, correlation measures association rather than causation and does not capture all possible nonlinear relationships.

12. Timestamp Feature Engineering

```
# =====
# 12. TIMESTAMP FEATURE ENGINEERING
# =====

df["Timestamp"] = pd.to_datetime(df["Timestamp"])

df["Hour"] = df["Timestamp"].dt.hour
df["Month"] = df["Timestamp"].dt.month
df["DayOfWeek"] = df["Timestamp"].dt.dayofweek

df.drop("Timestamp", axis=1, inplace=True)

display(df.head())
```

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Male	Country	Clicked on Ad	Hour	Month	DayOfWeek
0	68.95	35	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	0	Tunisia	0	0	3	6
1	80.23	31	68441.85	193.77	Monitored national standardization	West Jodi	1	Nauru	0	1	4	0

Insight

The original timestamp was transformed into Hour, Month, and DayOfWeek features. This allows the model to potentially capture temporal patterns in advertisement engagement while avoiding the direct use of a raw timestamp.

13. Analyze Clicks by Hour

```
# =====
# 13. CLICK RATE BY HOUR
# =====

hourly_ctr = (
    df.groupby("Hour")["Clicked on Ad"]
      .mean()
      .reset_index()
)

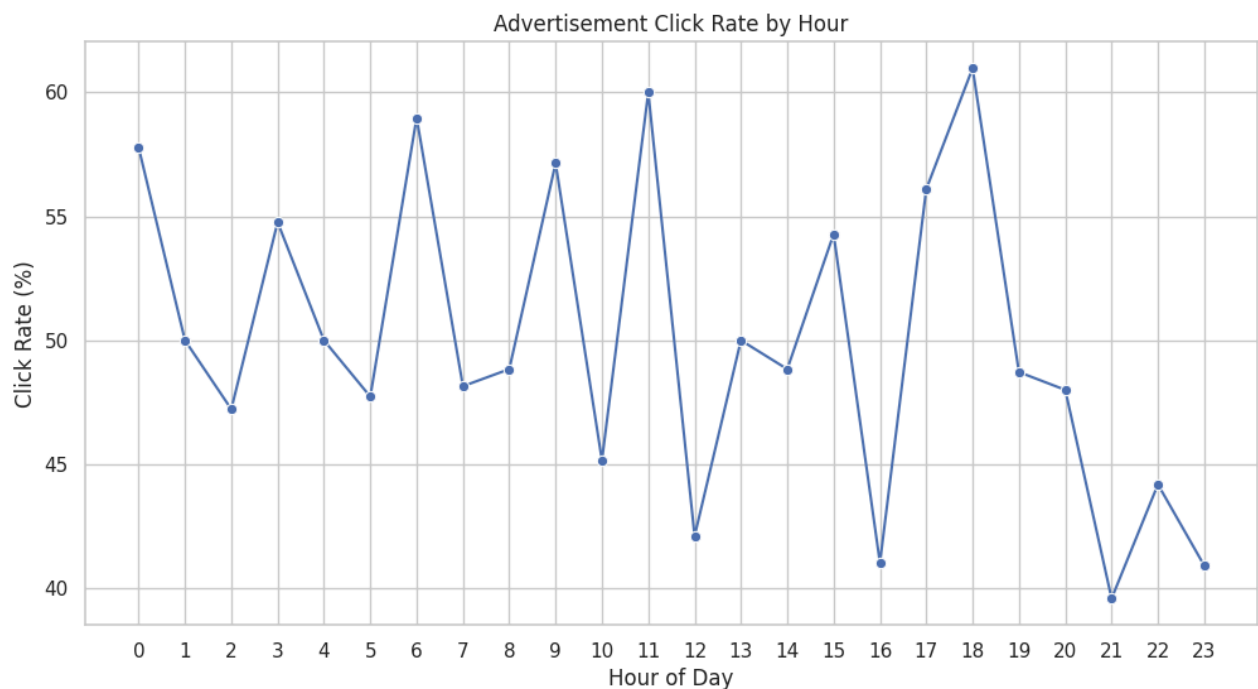
hourly_ctr["Clicked on Ad"] *= 100

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=hourly_ctr,
    x="Hour",
    y="Clicked on Ad",
    marker="o"
)

plt.title("Advertisement Click Rate by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Click Rate (%)")
plt.xticks(range(24))

plt.show()
```



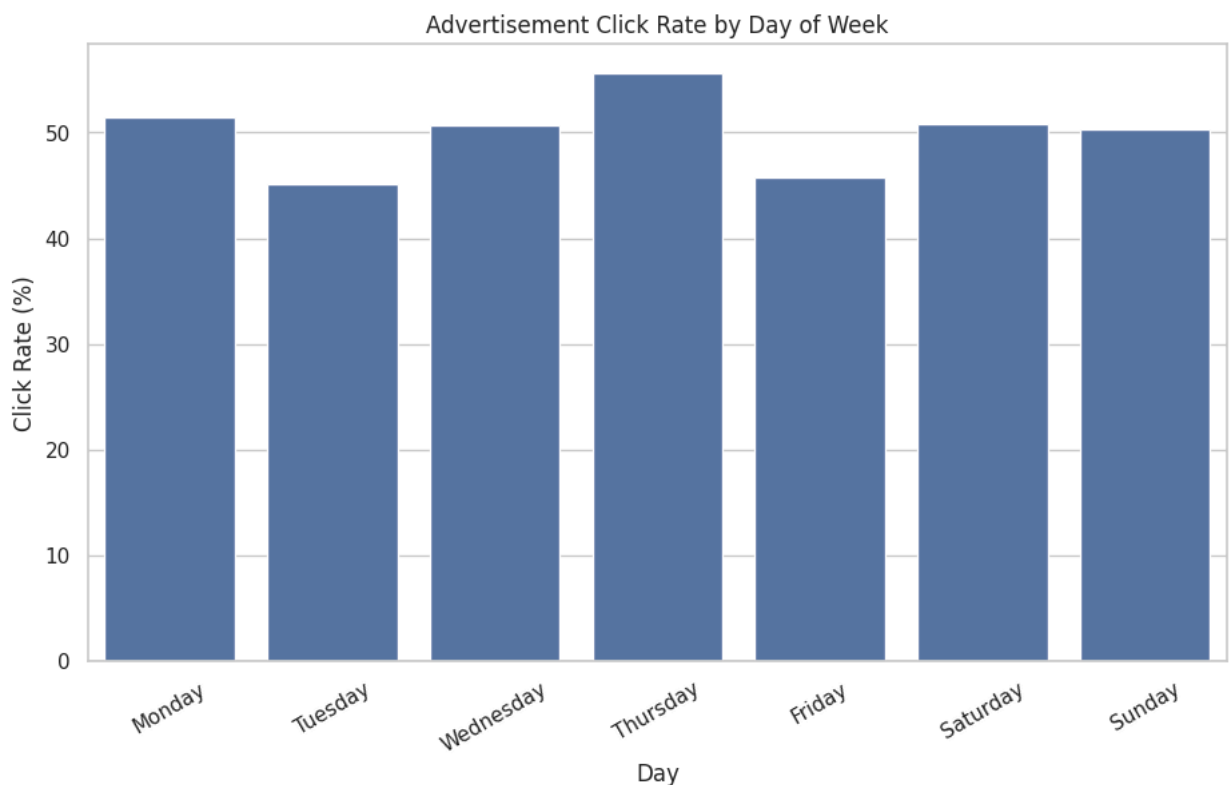
Insight

The hourly click-rate analysis investigates whether advertisement engagement varies throughout the day. If meaningful differences are observed, time-of-day can potentially contribute to more targeted advertising strategies.

14. Analyze Clicks by Day

```
# =====
# 14. CLICK RATE BY DAY OF WEEK
# =====
```

```
day_ctr = (  
    df.groupby("DayOfWeek")["Clicked on Ad"]  
        .mean()  
        .reset_index()  
)  
  
day_ctr["Clicked on Ad"] *= 100  
  
day_names = [  
    "Monday",  
    "Tuesday",  
    "Wednesday",  
    "Thursday",  
    "Friday",  
    "Saturday",  
    "Sunday"  
)  
  
day_ctr["Day"] = day_ctr["DayOfWeek"].map(  
    dict(enumerate(day_names))  
)  
  
plt.figure(figsize=(11, 6))  
  
sns.barplot(  
    data=day_ctr,  
    x="Day",  
    y="Clicked on Ad"  
)  
  
plt.title("Advertisement Click Rate by Day of Week")  
plt.xlabel("Day")  
plt.ylabel("Click Rate (%)")  
plt.xticks(rotation=30)  
  
plt.show()
```



Insight

Click-rate variation across weekdays can reveal whether engagement changes according to the day on which an advertisement is displayed. Such patterns can potentially support campaign scheduling decisions.

15. Outlier Detection

```
# =====
# 15. OUTLIER DETECTION USING IQR
# =====

def detect_outliers_iqr(data, columns):

    results = []

    for col in columns:

        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)

        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        count = (
            (data[col] < lower_bound) |
            (data[col] > upper_bound)
        ).sum()

        results.append({
            "Feature": col,
            "Q1": Q1,
            "Q3": Q3,
            "IQR": IQR,
            "Lower Bound": lower_bound,
            "Upper Bound": upper_bound,
            "Outlier Count": count
        })

    return pd.DataFrame(results)

outlier_summary = detect_outliers_iqr(
    df,
    numeric_features
)

display(outlier_summary)
```

	Feature	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count	
0	Daily Time Spent on Site	51.3600	78.5475	27.1875	10.57875	119.32875	0	
1	Age	29.0000	42.0000	13.0000	9.50000	61.50000	0	
2	Area Income	47031.8025	65470.6350	18438.8325	19373.55375	93128.88375	9	
3	Daily Internet Usage	138.8300	218.7925	79.9625	18.88625	338.73625	0	

Next steps: [Generate code with outlier_summary](#) [New interactive sheet](#)

Insight

The IQR method was used to systematically identify observations outside the typical range of each numerical variable. Importantly, an outlier is not automatically an error; it may represent a legitimate user with unusual browsing behavior. Therefore, outlier treatment should be based on business and data-quality considerations rather than removing every statistically unusual observation

16. Remove Irrelevant Columns

```
# =====
# 16. REMOVE IRRELEVANT HIGH-CARDINALITY FEATURES
# =====

columns_to_drop = [
    "Ad Topic Line",
    "City",
    "Country"
]

df_model = df.drop(
    columns=columns_to_drop,
    errors="ignore"
).copy()
```

```
print("Final modeling dataset shape:", df_model.shape)
```

```
Final modeling dataset shape: (1000, 9)
```

Insight

Ad Topic Line, City, and Country were excluded from the baseline Logistic Regression model to maintain a focused numerical feature set and avoid introducing high-cardinality categorical variables without an explicit encoding strategy. The model therefore concentrates on demographic, behavioral, and temporal features.

17. Features and Target

```
# =====  
# 17. DEFINE FEATURES AND TARGET  
# =====
```

```
X = df_model.drop("Clicked on Ad", axis=1)
```

```
y = df_model["Clicked on Ad"]
```

```
print("Feature Shape:", X.shape)  
print("Target Shape:", y.shape)
```

```
Feature Shape: (1000, 8)  
Target Shape: (1000,)
```

Insight

The dataset was divided into independent variables representing user characteristics and behavior, while Clicked on Ad was defined as the binary target variable.

18. Train-Test Split

```
# =====  
# 18. TRAIN-TEST SPLIT  
# =====  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

```
print("Training samples:", X_train.shape[0])  
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 800  
Testing samples: 200
```

Insight

The dataset was divided into 80% training data and 20% testing data. Stratification was applied to preserve the original proportion of click and non-click observations in both subsets, providing a more representative evaluation.

19. Feature Scaling

```
# =====  
# 19. FEATURE SCALING  
# =====
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Insight

Standardization was applied so that numerical features operate on comparable scales. The scaler was fitted only on the training data and then applied to the test data. This prevents information from the test set from influencing the preprocessing stage and avoids data leakage.

20. Train Logistic Regression

```
# =====
# 20. LOGISTIC REGRESSION
# =====

model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

model.fit(
    X_train_scaled,
    y_train
)

print("Model training completed successfully.")
```

Model training completed successfully.

Insight

Logistic Regression was trained to estimate the probability that a user will click an advertisement. Because the target has two possible outcomes—click or no click—the model is appropriate for binary classification and also provides interpretable coefficients for understanding feature influence.

21. Predictions

```
# =====
# 21. PREDICTIONS
# =====

y_pred = model.predict(X_test_scaled)

y_proba = model.predict_proba(X_test_scaled)[: , 1]

print("Predicted classes:")
print(y_pred[:10])

print("\nPredicted click probabilities:")
print(y_proba[:10])
```

Predicted classes:
[1 0 0 0 1 1 1 1 0 0]

Predicted click probabilities:
[0.99989271 0.0053368 0.02697929 0.01413257 0.99995792 0.99998917
0.9997805 0.99985659 0.04032034 0.01128553]

Insight

The model produces both a binary prediction and a probability of clicking. The probability output is particularly useful in advertising because users can potentially be prioritized according to their estimated click propensity rather than treating every user equally.

22. MODEL EVALUATION

```
# =====
# 22. MODEL EVALUATION
# =====

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)
```

```
roc_auc = roc_auc_score(y_test, y_proba)

average_precision = average_precision_score(
    y_test,
    y_proba
)

results = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score",
        "ROC-AUC",
        "Average Precision"
    ],
    "Score": [
        accuracy,
        precision,
        recall,
        f1,
        roc_auc,
        average_precision
    ]
})

display(results)
```

	Metric	Score	
0	Accuracy	0.980000	 
1	Precision	0.989796	
2	Recall	0.970000	
3	F1-Score	0.979798	
4	ROC-AUC	0.991500	
5	Average Precision	0.993452	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

Insight

Multiple evaluation metrics were used because no single metric completely describes classification performance. Accuracy measures overall correctness, precision measures the reliability of predicted clicks, recall measures how many actual clickers were identified, F1-score balances precision and recall, and ROC-AUC evaluates the model's ability to distinguish between click and non-click users across classification thresholds.

23. CLASSIFICATION REPORT

```
# =====
# 23. CLASSIFICATION REPORT
# =====

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Click", "Click"]
    )
)
```

	precision	recall	f1-score	support
No Click	0.97	0.99	0.98	100
Click	0.99	0.97	0.98	100
accuracy			0.98	200
macro avg	0.98	0.98	0.98	200
weighted avg	0.98	0.98	0.98	200

24. CONFUSION MATRIX

```
# =====
# 24. CONFUSION MATRIX
# =====

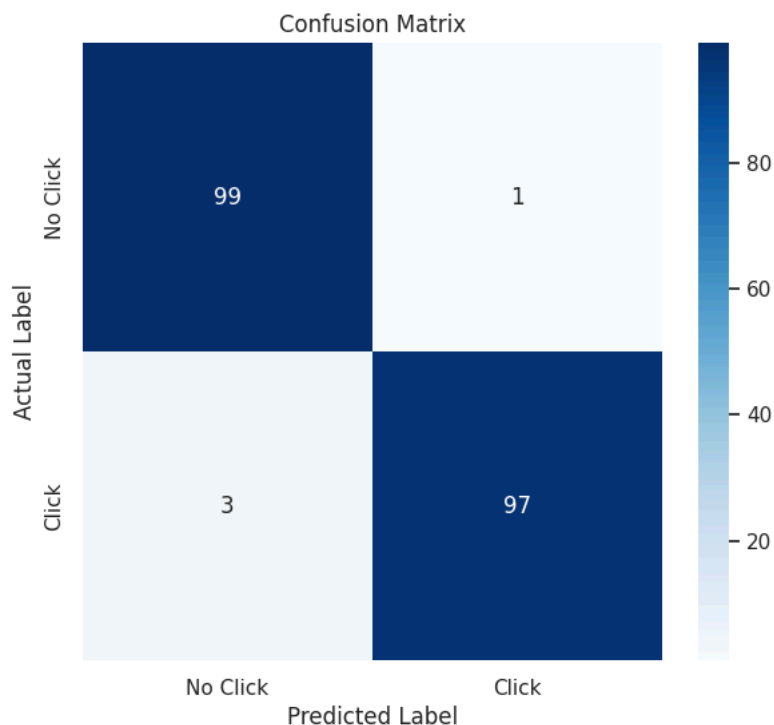
cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(7, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Click", "Click"],
    yticklabels=["No Click", "Click"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

plt.show()
```



Insight

The classification report provides class-level precision, recall, and F1-score. This allows the performance of click and non-click predictions to be examined separately rather than relying only on an overall accuracy score.

25. ROC-AUC CURVE

```
# =====
# 25. ROC-AUC CURVE
# =====

fpr, tpr, thresholds = roc_curve(
    y_test,
    y_proba
)

plt.figure(figsize=(9, 6))

plt.plot(
    fpr,
    tpr,
```

```

    label=f"Logistic Regression (AUC = {roc_auc:.3f})"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

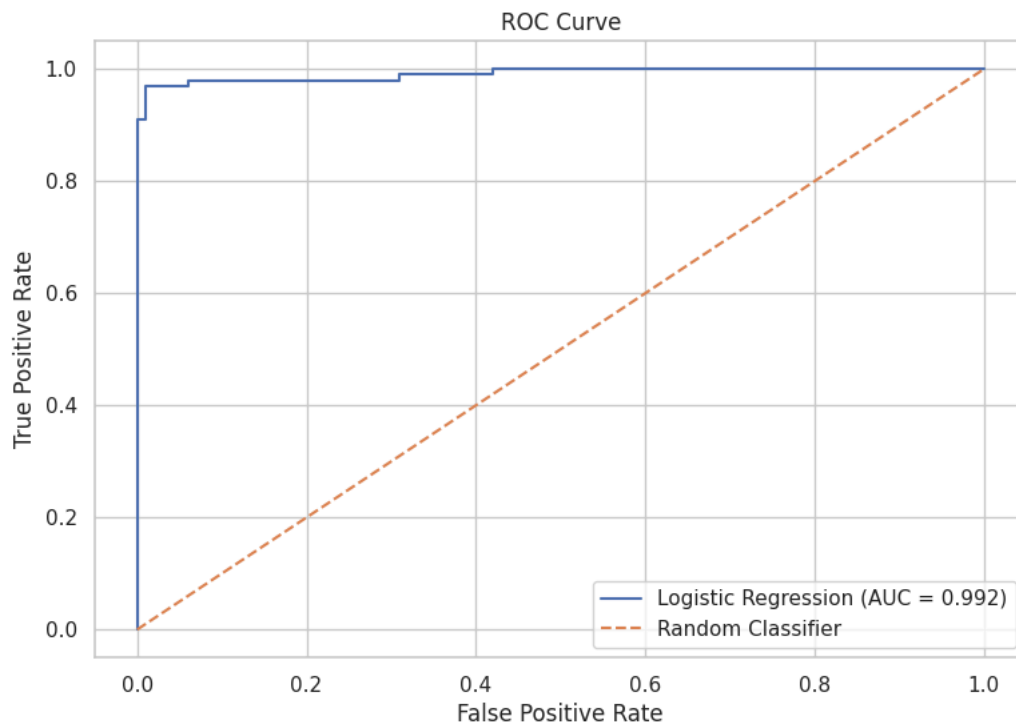
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.show()

```



Insight

The confusion matrix separates predictions into true positives, true negatives, false positives, and false negatives. In an advertising context, false negatives represent users who were likely to click but were not identified by the model, while false positives represent users predicted to click who ultimately did not.

26. PRECISION-RECALL CURVE

```

# =====
# 26. PRECISION-RECALL CURVE
# =====

precision_values, recall_values, pr_thresholds = precision_recall_curve(
    y_test,
    y_proba
)

plt.figure(figsize=(9, 6))

plt.plot(
    recall_values,
    precision_values,
    label=f"Logistic Regression (AP = {average_precision:.3f})"
)

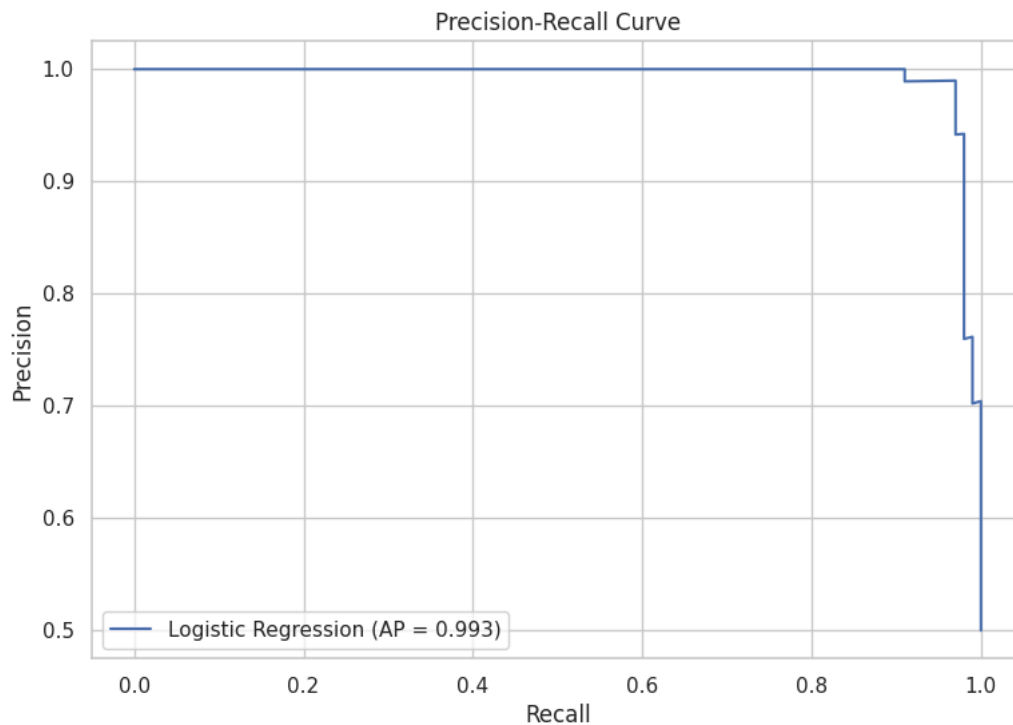
plt.xlabel("Recall")
plt.ylabel("Precision")

plt.title("Precision-Recall Curve")

```

```
plt.legend()
```

```
plt.show()
```



Insight

The ROC curve illustrates the trade-off between the true-positive rate and false-positive rate across different probability thresholds. ROC-AUC summarizes the model's ability to distinguish between users who click and users who do not.

27. FEATURE COEFFICIENT ANALYSIS

```
# =====  
# 27. FEATURE COEFFICIENT ANALYSIS  
# =====  
  
feature_importance = pd.DataFrame({  
    "Feature": X.columns,  
    "Coefficient": model.coef_[0]  
})  
  
feature_importance["Absolute Coefficient"] = (  
    feature_importance["Coefficient"].abs()  
)  
  
feature_importance = feature_importance.sort_values(  
    "Coefficient",  
    ascending=False  
)  
  
display(feature_importance)
```

	Feature	Coefficient	Absolute Coefficient
1	Age	1.248907	1.248907
7	DayOfWeek	0.310661	0.310661
6	Month	-0.018086	0.018086
5	Hour	-0.142959	0.142959
4	Male	-0.224455	0.224455
2	Area Income	-1.544383	1.544383
3	Daily Internet Usage	-2.340795	2.340795
0	Daily Time Spent on Site	-2.884374	2.884374

Next steps: [Generate code with feature_importance](#) [New interactive sheet](#)

Insight

The precision-recall curve evaluates the trade-off between identifying more potential clickers and maintaining the accuracy of click predictions. This perspective is useful for advertising because the appropriate threshold can depend on the relative cost of targeting users who are unlikely to engage versus missing potential customers.

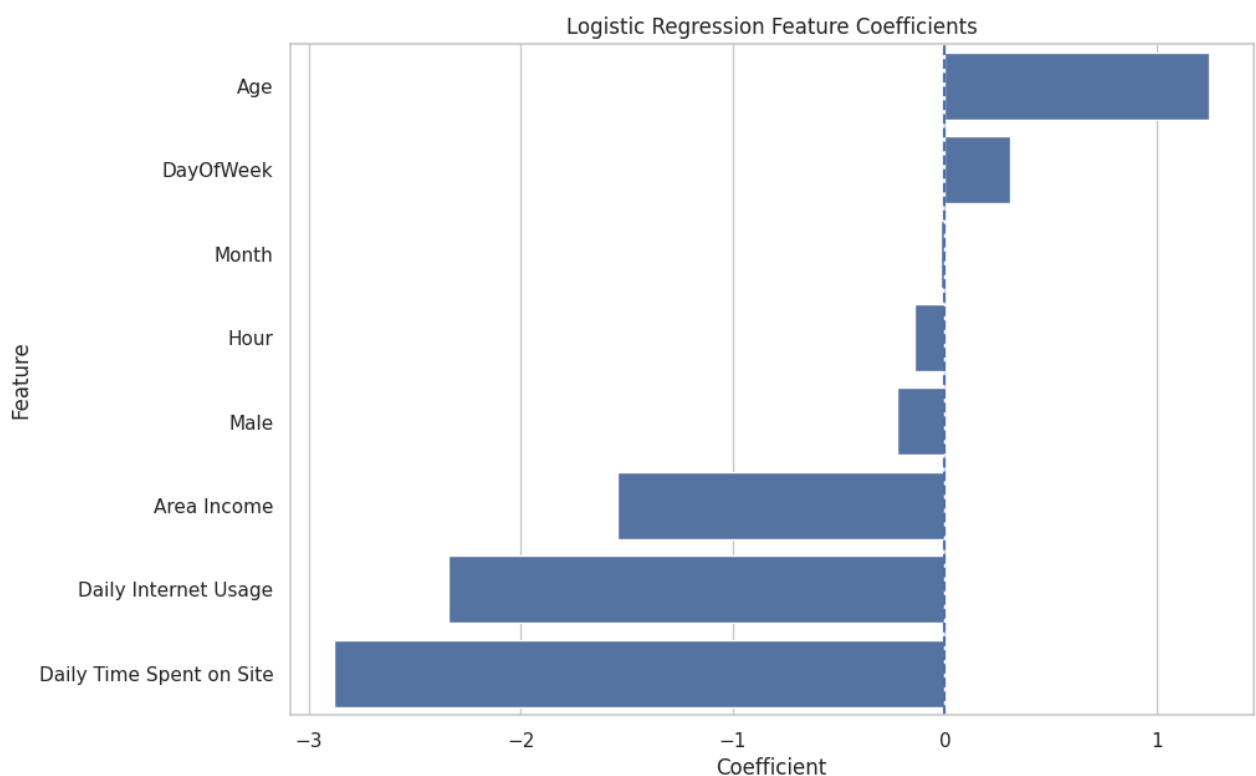
```
plt.figure(figsize=(10, 7))

sns.barplot(
    data=feature_importance,
    x="Coefficient",
    y="Feature"
)

plt.axvline(
    0,
    linestyle="--"
)

plt.title("Logistic Regression Feature Coefficients")
plt.xlabel("Coefficient")
plt.ylabel("Feature")

plt.show()
```



Insight

Logistic Regression coefficients provide an interpretable view of how each standardized feature is associated with the predicted probability of an advertisement click. Positive coefficients indicate a positive association with the click class, while negative coefficients indicate a negative association. Because the features were standardized, the coefficient magnitudes can also be compared more meaningfully across variables.

28. ODDS RATIO ANALYSIS

```
# =====
# 28. ODDS RATIO ANALYSIS
# =====

feature_importance["Odds Ratio"] = np.exp(
```

```

        feature_importance["Coefficient"]
    )

display(
    feature_importance[
        ["Feature", "Coefficient", "Odds Ratio"]
    ]
    .sort_values("Odds Ratio", ascending=False)
)

```

	Feature	Coefficient	Odds Ratio	
1	Age	1.248907	3.486530	
7	DayOfWeek	0.310661	1.364326	
6	Month	-0.018086	0.982076	
5	Hour	-0.142959	0.866790	
4	Male	-0.224455	0.798952	
2	Area Income	-1.544383	0.213443	
3	Daily Internet Usage	-2.340795	0.096251	
0	Daily Time Spent on Site	-2.884374	0.055890	

Insight

The odds ratio provides an interpretable representation of Logistic Regression coefficients. An odds ratio above 1 indicates a positive association with click odds, while an odds ratio below 1 indicates a negative association, holding the other model features constant.

29. ACTUAL VS PREDICTED CLASS DISTRIBUTION

```

# =====
# 29. ACTUAL VS PREDICTED CLASS DISTRIBUTION
# =====

comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.countplot(
    data=comparison,
    x="Actual",
    ax=axes[0]
)

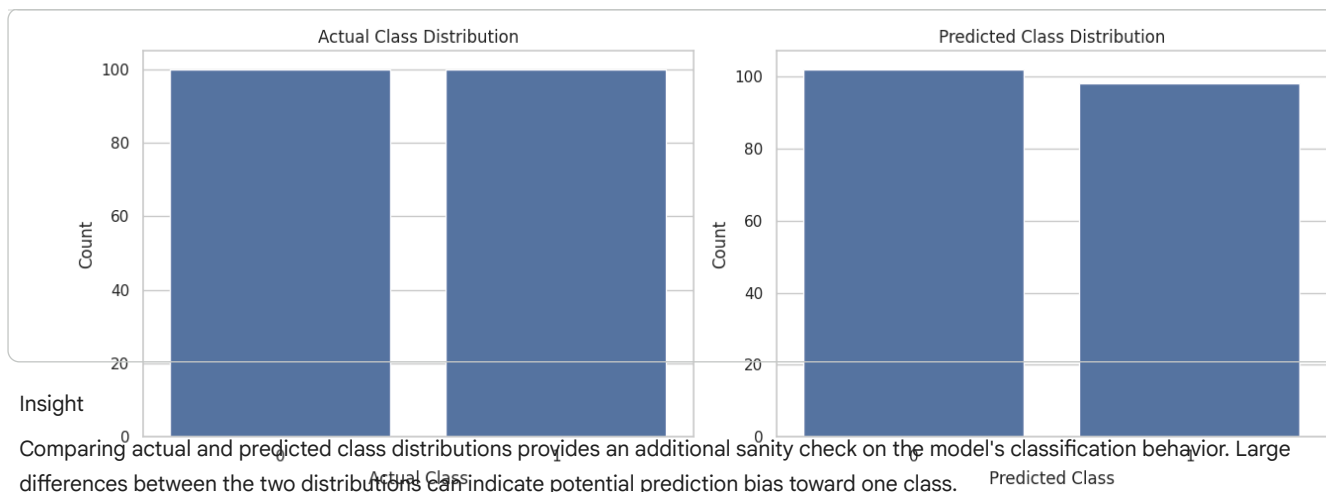
axes[0].set_title("Actual Class Distribution")
axes[0].set_xlabel("Actual Class")
axes[0].set_ylabel("Count")

sns.countplot(
    data=comparison,
    x="Predicted",
    ax=axes[1]
)

axes[1].set_title("Predicted Class Distribution")
axes[1].set_xlabel("Predicted Class")
axes[1].set_ylabel("Count")

plt.tight_layout()
plt.show()

```



30. FINAL MODEL SUMMARY

```
# =====  
# 30. FINAL MODEL SUMMARY  
# =====  
  
print("=" * 50)  
print("FINAL MODEL PERFORMANCE")  
print("=" * 50)  
  
print(f"Accuracy          : {accuracy:.4f}")  
print(f"Precision         : {precision:.4f}")  
print(f"Recall            : {recall:.4f}")  
print(f"F1-Score          : {f1:.4f}")  
print(f"ROC-AUC           : {roc_auc:.4f}")  
print(f"Average Precision : {average_precision:.4f}")
```

```
=====  
FINAL MODEL PERFORMANCE  
=====  
Accuracy          : 0.9800  
Precision         : 0.9898  
Recall            : 0.9700  
F1-Score          : 0.9798  
ROC-AUC           : 0.9915  
Average Precision : 0.9935
```

Overall Insight

The project developed a Logistic Regression model to predict whether a user is likely to click an online advertisement using demographic, behavioral, and temporal information. The analysis included data-quality checks, exploratory data analysis, correlation analysis, feature engineering, outlier investigation, feature scaling, and binary classification.

The model was evaluated using accuracy, precision, recall, F1-score, ROC-AUC, and Average Precision to provide a comprehensive assessment of predictive performance. The confusion matrix and ROC/Precision-Recall curves were also examined to understand the model's classification behavior across different decision thresholds.